

8- RAID & Distributed Storage

Unat Teksen

March 30-31, 2026

1 Introduction: Beyond a Single Disk

1.1 Why go beyond a single disk?

- A single hard disk has two fundamental limits: a **fixed capacity ceiling** and a **fixed performance ceiling**. If you need more, you are stuck.
- To make multiple physical disks appear as one (e.g., 10×1 TB drives $\rightarrow$ single 10 TB drive, 10×20 MB/s $\rightarrow$ 200 MB/s), we use RAID.
- **RAID (Redundant Array of Independent Disks)**: Proposed in the 1980s by Patterson et al. (originally "Inexpensive Disks").
- **Core Promise**: Combines multiple independent physical disks into a single large, high-performance logical disk, hiding all internal complexity from the OS and applications.
- **Contrast: JBOD (Just a Bunch of Disks)** $\rightarrow$ Each disk is a completely separate device with its own mount point (no pooling, no transparency).

1.2 The Two Fundamental Techniques in RAID

Everything in RAID builds on exactly two orthogonal ideas:

1. **Data Striping**: Used to **improve performance**. Data is split across multiple disks according to a round-robin algorithm. Large files are served by multiple disks simultaneously, multiplying effective throughput.
 - *Performance benefits*: Multiple independent I/O requests spread naturally across all disks (reducing queue length), and single large requests are parallelized.
2. **Redundancy**: Used to **improve reliability**. Error-correcting information is stored so that if one disk fails, lost data can be reconstructed.

Key Vocabulary:

- **Stripe Unit (Chunk Size)**: The amount of data written to a single disk before moving to the next (bit, byte, or typically 64 KB block).
- **Stripe Width**: The number of disks participating in the striping algorithm.

1.3 The Reliability Paradox

- **The Tension:** The more disks you add to improve capacity/performance, the more likely at least one of them fails. Mathematically inescapable.
- **MTTF (Mean Time To Failure):** The expected time until a component fails, assuming continuous operation.
- **Example:** If 1 disk has MTTF of 200,000 hours (23 years), an array of 100 disks has a combined MTTF of $200,000/100 = 2,000$ hours (~ 3 months). You gain 100x capacity but lose almost all reliability!
- **Result:** Redundancy is mandatory. Trade-off $\rightarrow$ every write must also update redundant information, making writes slower than a non-redundant system.
- **Data Reconstruction Example:** Disks hold 10, 8, 2. A fourth disk holds the sum (20). If Disk 1 fails: $20 - (10 + 2) = 8$. This is the foundation of RAID 4, 5, 6.

1.4 What RAID Looks Like from the Outside

- **Transparency:** Completely transparent to the OS and applications. OS sees a single logical disk and issues normal block reads/writes.
- **RAID Controller:** The hardware (or software) layer that presents multiple physical disks as a single logical device. It acts as a full computer system (has its own CPU, RAM, NVRAM) to manage disks, perform striping, compute parity, and handle recovery.

Main Takeaway

RAID's transparency is its greatest engineering achievement. Applications need zero modification to benefit from it.

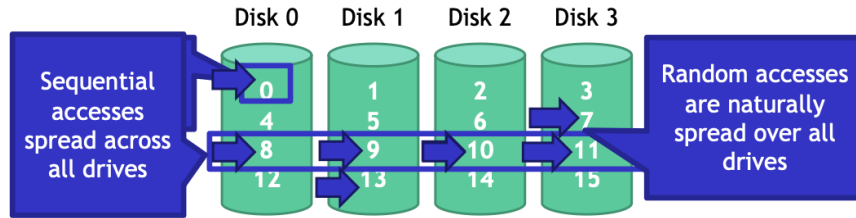
2 RAID Levels Deep Dive

Overview of Levels Required for Course:

- **RAID 0:** Striping only, no redundancy.
- **RAID 1:** Mirroring only.
- **RAID 0+1 & 1+0:** Nested combinations.
- **RAID 4:** Block striping with a dedicated parity disk.
- **RAID 5:** Block striping with distributed parity.
- **RAID 6:** Distributed parity with tolerance for 2 simultaneous failures.

2.1 RAID 0: Striping, No Redundancy

- Spreads data across N disks in round-robin fashion (Block 0 $\rightarrow$ Disk 0, Block 1 $\rightarrow$ Disk 1, etc.).



- **Addressing formulas:**

- Which disk? $\rightarrow \text{Logical_Block_Number} \pmod{\text{Number_of_disks}}$
- Which offset? $\rightarrow \text{Logical_Block_Number} / \text{Number_of_disks}$
- Example: Block 11 in a 4-disk array $\rightarrow \text{Disk } 11 \pmod{4} = 3$. Offset $11/4 = 2$ (the 3rd physical block on Disk 3, counting from 0).

- **Chunk Size:** Smaller chunks = more parallelism (I/O spans more disks). Larger chunks = reduced seek times (more file lives on one disk). 64 KB is a typical balance.

Property	Value / Description
Usable Capacity	N (100% of all disks)
Reliability	Zero (any single failure loses everything)
MTTDL	MTTF of a single disk / N
Write Performance	Best possible (no parity overhead, full parallelism)
Use Case	Non-critical data where performance matters most

Exam Focus

RAID 0 has best performance and worst reliability. Single disk failure = total loss.
 $\text{MTTF}_{\text{RAID0}} = \text{MTTF}_{\text{disk}} / N$.

2.2 RAID 1: Mirroring

- Writes every block to two (or more) disks simultaneously. Exact copy on each. Minimum 2 drives.
- **Read behavior:** Served from whichever copy has the shorter queue, seek time, and rotational delay (improves read throughput).
- **Write behavior:** Every write must go to both disks. No parity computation, but duplication makes writes slower than a single disk.
- **Theoretical Note:** Can mirror to > 2 disks. With 3 copies, a voting mechanism can detect silent corruption. (Never done in practice due to high cost).

2.3 Nested Levels: RAID 0+1 vs RAID 1+0

Notation **RAID x+y**: Apply RAID x within each group $\rightarrow$ then apply RAID y across groups.

Property	Value / Description
Usable Capacity	$N/2$ (50% — half is always mirrors)
Reliability	Excellent — 1 drive can always fail (with luck, $N/2$ drives)
Write Performance	Good (no parity calculation) but slower than RAID 0
Use Case	Critical data where losing anything is unacceptable

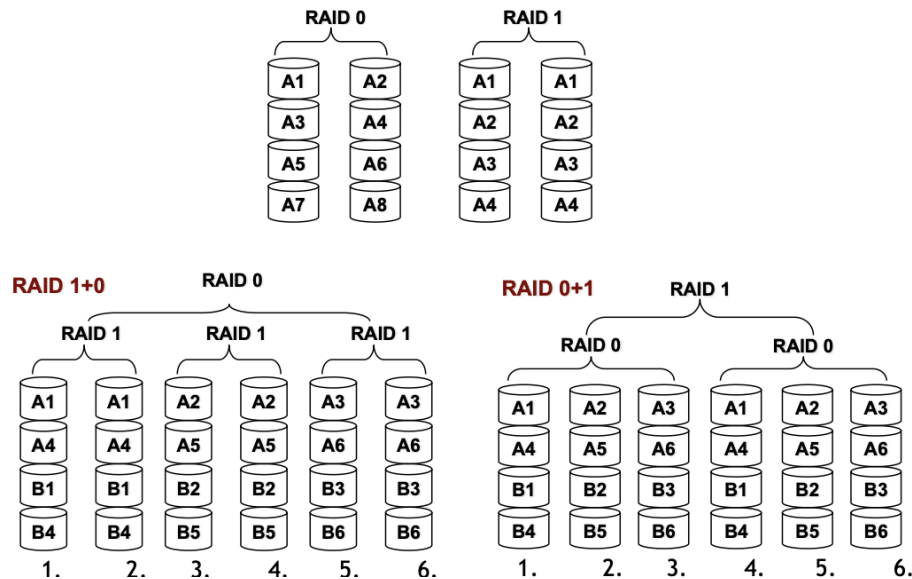


Figure 1: Enter Caption

- **RAID 0+1 (Stripe first, Mirror second):**

- Stripe across pairs (RAID 0), then mirror the two striped groups (RAID 1).
- **Critical Weakness:** If 1 disk in stripe group A fails, the **entire group A** is considered failed. System survives as pure RAID 0 → one more failure anywhere = total data loss. Low effective tolerance.

- **RAID 1+0 (Mirror first, Stripe second):**

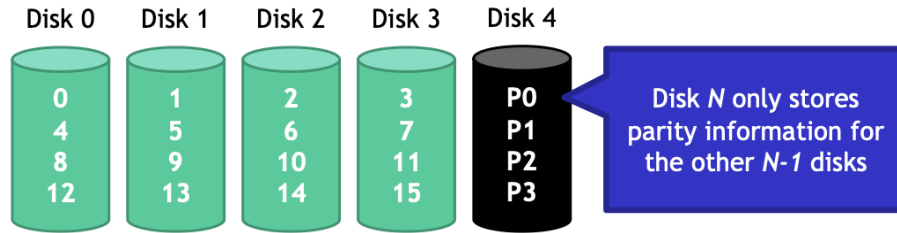
- Mirror each pair (RAID 1), then stripe across the mirrored pairs (RAID 0).
- **Strength:** Because each pair is independently mirrored, multiple disks can fail simultaneously as long as no two failures are from the same mirror pair. **Much more robust.**

Exam Focus

Order matters! Both have capacity $N/2$ and identical R/W performance. However, **RAID 1+0 has higher fault tolerance**. RAID 1+0 is always preferred for critical workloads.

2.4 The Consistent Update Problem

- **Problem:** Mirrored writes must be atomic. If power fails mid-write, one copy is updated and the other is stale.



- **Solution: Write-Ahead Log (WAL).** The controller records pending writes in battery-backed NVRAM. On reboot, recovery reads the log and re-applies incomplete writes to sync mirrors.

2.5 RAID 4: Dedicated Parity Disk

- Adds redundancy by dedicating one entire disk to parity. Data is striped across $N - 1$ disks.
- **XOR Parity:** XOR all data blocks together = parity block. If one block is lost, XOR all remaining blocks with parity to recover it. (Very cheap to compute).
- **Parity Update Strategies:**
 1. **Additive Parity:** Read all $N - 1$ data blocks $\rightarrow$ XOR together with new block $\rightarrow$ write new parity.
 2. **Subtractive Parity (Efficient):** Requires only 2 reads and 2 writes (4 ops total).

$$P_{new} = C_{old} \text{ XOR } C_{new} \text{ XOR } P_{old}$$

- **The Fatal Flaw (Small Write Problem):** No matter which data disk you write to, you must ALWAYS update the parity disk. **All random writes serialize on that one disk.**

Property	Value / Description
Usable Capacity	$N - 1$ (one disk is always parity)
Reliability	1 drive can fail (recoverable)
Sequential Read	Very good (parallelized across $N - 1$ disks)
Random Write	Terrible (bottlenecks on the single parity disk)

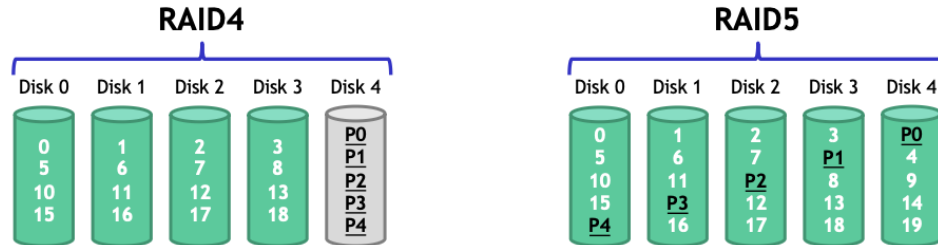
2.6 RAID 5: Distributed Parity

- Solves RAID 4's bottleneck by **rotating** which disk holds the parity block for each stripe. No single disk is the bottleneck.
- Random write process is identical (4 disk ops via subtractive parity), but load is spread evenly across all disks.

2.7 RAID 6: Double Parity

- Stores **two independent parity blocks** (P and Q) per stripe, distributed across all disks. Uses Reed-Solomon erasure codes (not just XOR).

Property	Value / Description
Usable Capacity	$N - 1$ (same as RAID 4)
Reliability	1 drive can fail
Read Performance	Good (parallelized)
Random Write	Much better than RAID 4 (distributed evenly)
Rebuild Perf.	Poor (requires reading all remaining disks)
Use Case	Databases, transaction apps (The Sweet Spot)



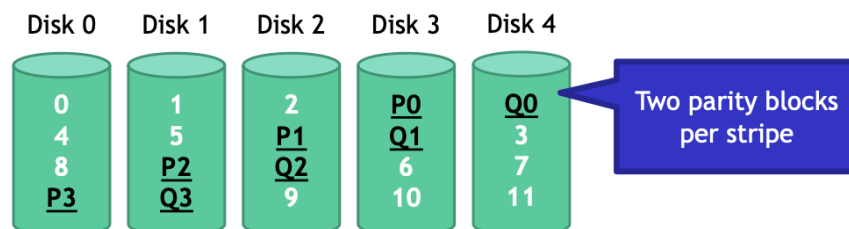
- Survives **two simultaneous disk failures**. Minimum configuration: 6 disks (4 data + 2 parity).
- Cost:** Every write requires **6 disk accesses** (Read old block, old P, old Q → Write new block, new P, new Q). Very slow writes.

2.8 RAID Characteristics Summary Table

Rules of thumb: Max performance/capacity → RAID 0. Max recovery → RAID 1/1+0 or 6. Best balance → RAID 5.

2.9 Hot Spares & Controller Types

- Hot Spare:** An idle disk installed in the array. When a drive fails, the controller **automatically** begins rebuilding onto the hot spare without human intervention, minimizing the window of vulnerability.
- Hardware RAID:** Faster, more reliable, has battery-backed cache. Downside: Lock-in (proprietary formats prevent migrating to different controllers).
- Software RAID:** Implemented by OS (e.g., Linux mdadm). Portable and cheap, but lower performance and highly vulnerable to the consistent update problem (OS lacks battery-backed WAL).



Property	Value / Description				
Usable Capacity	$N - 2$				
Reliability	Excellent (2 simultaneous failures tolerated)				
Write Performance	Poor (6 accesses per write)				
Use Case	Critical data requiring max redundancy				

RAID	Capacity	Reliability	R/W Performance	Rebuild	Best for
0	100%	None	Very good / very good	Good	Non-critical, fast I/O
1	50%	Excellent	Very good / good	Good	Critical data
5	$(N - 1)/N$	Good	Good / fair	Poor	Databases, general
6	$(N - 2)/N$	Excellent	Very good / poor	Poor	Critical + max redundancy
1+0	50%	Excellent	Very good / good	Good	Critical + performance

3 Reliability Calculations (MTTF Formulas)

3.1 Foundational Assumptions

- **Constant failure rate:** predictable over time.
- **Exponentially distributed** time to failure.
- **Independent failures:** one disk failing doesn't cause another.
- **MTTF (Mean Time To Failure):** Expected time before a SINGLE disk fails.
- **MTTR (Mean Time To Repair):** Time to replace a failed disk AND fully rebuild the array. (This is the **Danger Window**).

3.2 The General Master Formula

Applies to ALL redundant RAID levels:

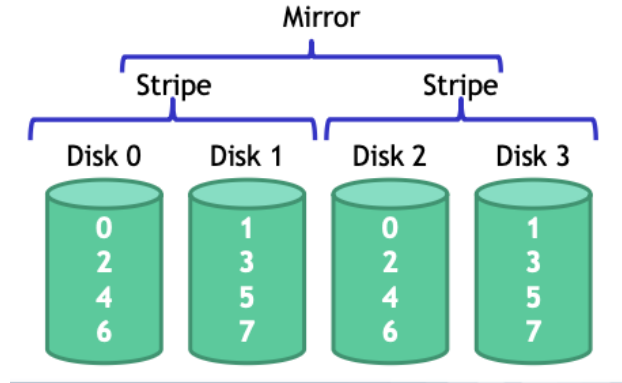
$$\text{MTTF}_{\text{RAID}} = \frac{\text{MTTF}_{\text{singleDisk}}}{N} \times \frac{1}{\text{Probability}_{\text{additionalCriticalFailuresInMTTR}}}$$

- **Left term:** Failure rate of the whole array (first failure happens N times faster).
- **Right term:** Redundancy benefit. The probability that a critical second failure happens BEFORE the first is repaired.
- *Takeaway:* RAID doesn't eliminate failure, it just makes the probability of simultaneous critical failures during the repair window astronomically low.

3.3 Formula Derivations

RAID 0 (Unprotected Array):

- No redundancy. Fails the moment the first disk fails.



-

$$\text{MTTF}_{\text{RAID0}} = \frac{\text{MTTF}_{\text{singleDisk}}}{N}$$

RAID 1 (Mirroring):

- Data lost only if the mirror fails during repair window. Prob = $(1/\text{MTTF}) \times \text{MTTR}$.

-

$$\text{MTTF}_{\text{RAID1}} = \frac{(\text{MTTF}_{\text{singleDisk}})^2}{N \times \text{MTTR}}$$

RAID 0+1 (Stripe then Mirror):

- 1 failure knocks out an entire group. Any of the G disks in the surviving group failing = data loss. Prob = $(G/\text{MTTF}) \times \text{MTTR}$.

-

$$\text{MTTF}_{\text{RAID0+1}} = \frac{(\text{MTTF}_{\text{singleDisk}})^2}{N \times G \times \text{MTTR}}$$

- *Note the G in the denominator makes it less reliable than 1+0.*

RAID 1+0 (Mirror then Stripe):

- Only 1 specific disk can cause data loss (the exact mirror of the failed disk).

-

$$\text{MTTF}_{\text{RAID1+0}} = \frac{(\text{MTTF}_{\text{singleDisk}})^2}{N \times \text{MTTR}}$$

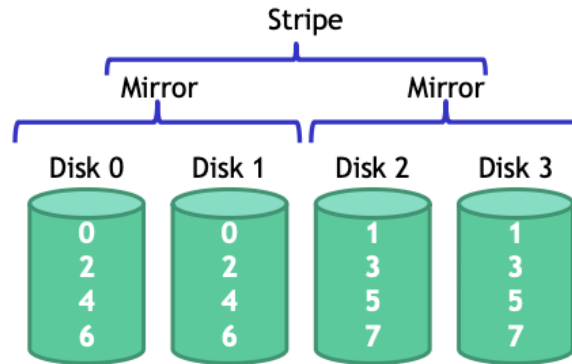
- *Identical to RAID 1 formula. Much better than 0+1.*

RAID 4 & RAID 5:

- Require 2 failures. Any of the remaining $(N - 1)$ disks failing causes data loss.

-

$$\text{MTTF}_{\text{RAID4}} = \text{MTTF}_{\text{RAID5}} = \frac{(\text{MTTF}_{\text{singleDisk}})^2}{N \times (N - 1) \times \text{MTTR}}$$



RAID 6:

- Tolerates 2 failures. Data lost if 3rd disk fails while both replacements are in progress (overlapping window $MTTR/2$).
- Prob 2nd = $((N - 1)/MTTF) \times MTTR$. Prob 3rd = $((N - 2)/MTTF) \times (MTTR/2)$.
-

$$MTTF_{RAID6} = \frac{2 \times (MTTF_{\text{singleDisk}})^3}{N \times (N - 1) \times (N - 2) \times MTTR^2}$$

Exam Focus

MTTF Summary for Exams

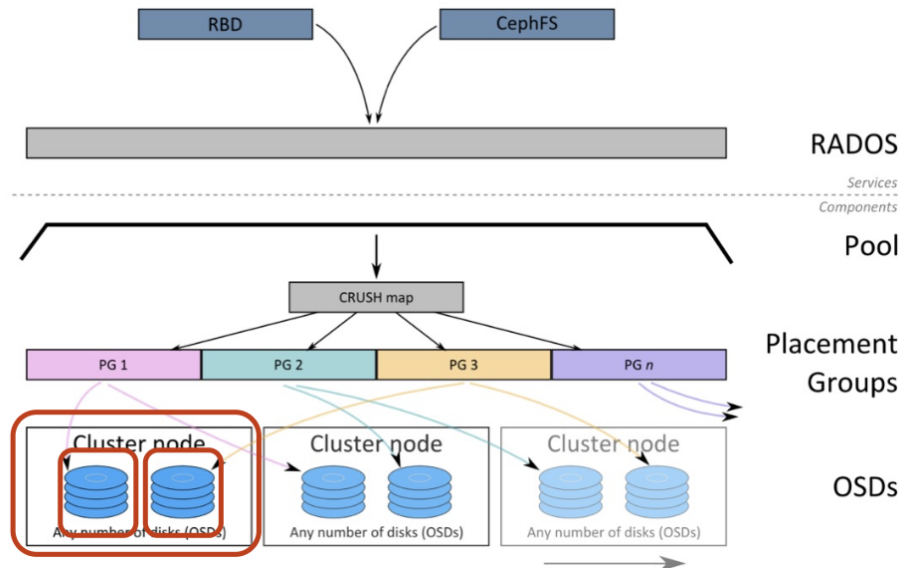
- **RAID 0:** $MTTF/N$
- **RAID 1+0:** $MTTF^2/(N \times MTTR)$
- **RAID 0+1:** $MTTF^2/(N \times G \times MTTR)$
- **RAID 4/5:** $MTTF^2/(N \times (N - 1) \times MTTR)$
- **RAID 6:** $2 \times MTTF^3/(N \times (N - 1) \times (N - 2) \times MTTR^2)$

Longer MTTR (repair time) = worse reliability. RAID 6 uniquely has the factor of 2, MTTF cubed, and MTTR squared.

4 From RAID to Distributed Storage

4.1 What is BLOB Storage?

- **BLOB (Binary Large Object):** Large chunks of unstructured binary data (images, videos, backups, ML models) stored by cloud hyperscalers (AWS, Azure, GCP).
- **General Goal:** Store massive volumes at planetary scale with high durability, global accessibility, and cost efficiency.
- **Key Properties:**
 - **Highly fault tolerant:** Geographically distributed. Even an entire data center going offline won't cause data loss.



- **Relatively high performance:** Served by CDN (physically closest node). Slower than local SAN due to Internet latency.
- **No overhead:** Hardware, backups, maintenance handled by provider.

4.2 How does RAID fit in the cloud?

- **RAID 0** is used inside cloud instances for **ephemeral (temporary)** high-performance storage (caches, ML pipelines).
- BLOB storage itself **does NOT** use traditional RAID. It distributes data using:
 1. **Erasure Coding:** Mathematically similar to RAID 5/6 but applied across entire servers/data centers.
 2. **Multi-replica strategies:** Keeping multiple full copies across geographic nodes.

5 CEPH: Distributed Storage System

5.1 What is CEPH?

- **CEPH:** A free, open-source distributed storage platform. Provides object, block, and file-level storage from a unified system.
- **Goals:** Completely distributed (no central bottleneck), **no single point of failure**, scalable to the exabyte level (10^{18} bytes).

5.2 Architecture (Two Levels)

- **Physical Level:** Interconnected machines called **CEPH nodes**. Each holds one or more actual storage disks.

- **Logical Level:** Each disk is an **OSD (Object Storage Device)**, the logical unit CEPH uses to read/write/track data.
- **Architecture Stack (Top to Bottom):**
 - **RBD / CephFS:** Interfaces exposed to applications.
 - **RADOS:** Core distributed object store underneath.
 - **Pool:** Logical grouping of capacity.
 - **Placement Groups (PGs):** Intermediate grouping of OSDs for data location.
 - **OSDs:** The actual disks at the bottom.

5.3 CEPH vs RAID Differences

- **RAID:** 4-16 disks in one machine, microsecond transfer, failures are disk-only.
- **CEPH:** Thousands of disks across machines/locations. **Network transfer times are NOT negligible** (ms to sec). Intermittent failures due to network issues.
- *Result:* CEPH requires far more redundancy than RAID.

5.4 CEPH Reliability Strategies

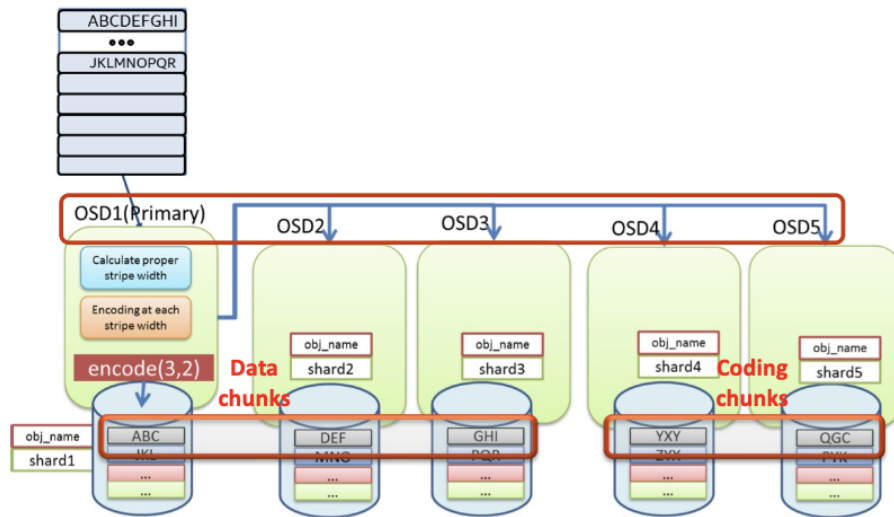
1. **Replication (like RAID 1):** Keeps multiple full copies. Uses more replicas (typically 3) because network failures are highly probable.
2. **Erasur Coding (like RAID 5/6):** Stores data chunks + parity chunks. Requires more extra units than RAID 5/6.

5.5 Resource Location (Placement Groups & CRUSH Map)

- **Problem:** Finding a file across thousands of servers.
- **Solution:** CEPH partitions all nodes/OSDs into **Placement Groups (PGs)**.
- From the **filename alone**, CEPH computes (via the **CRUSH map algorithm**) which PG the file belongs to, immediately revealing the OSDs.
- *Power:* Any node can compute the location independently. **No central directory server = no single point of failure.**

5.6 CEPH Operations (Replication & Erasure Coding)

- **Writing with Replication:** App writes to **primary OSD** → forwards copies to other OSDs in PG → acknowledged only when all confirm.
- **Reading with Replication:** Served by the closest/least loaded OSD. Auto-fallback to replicas if primary fails.
- **Writing with Erasure Coding:** File divided into **data chunks**, primary OSD computes **coding chunks** (parity) → distributed across all OSDs in PG.



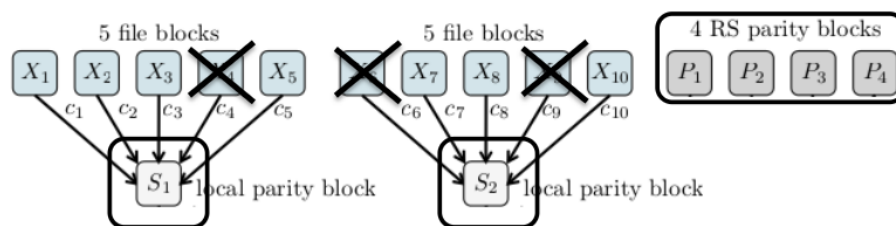
- **Reading with Erasure Coding:** Only data chunks are contacted (faster). Coding chunks are only touched during failure recovery.

5.7 Hierarchical Erasure Coding

- **Problem:** Standard erasure coding recovery is very expensive over a network (must contact ALL nodes).
- **Solution:** **Hierarchical Coding** uses two levels:
 1. **Local parity blocks (S1, S2):** Covers small local groups. If one block fails, only that local group participates in recovery (Fast).
 2. **Global parity blocks (P1-P4):** Reed-Solomon covering the whole dataset. Used ONLY when multiple failures span multiple local groups.
- *Example (X1-X10 blocks):* X4 fails → only local S1 recovers it. X4 + X6 fail → S1 recovers X4, S2 recovers X6. X4 + X6 + X9 fail → all blocks (S1, S2, P1-P4) reconstruct.

5.8 Duplication Limits: A Practical Warning

- Redundancy layers can **compound catastrophically**.
- *Example:* Hadoop (3x replicas) × CEPH (3x replication) × Local RAID 1 node (2x) = **18 copies of every byte!**
- **Result:** Only 1/18th of storage holds real data. Writes are 18x slower. Massive waste and terrible performance.
- **Takeaway:** Design systems holistically. Use erasure coding at CEPH level instead of compounding CEPH replication + local RAID.



All redundancy blocks (S_1, S_2, P_1, P_2, P_3 and P_4) are used to reconstruct X_4, X_6 and X_9 .